Information Sciences 753 (2026) 123619

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Privacy-preserving federated learning via secret sharing and multi-key homomorphic encryption

Yuntao Wang [a,*], Fumiya Inoue [a], Yujie Gu [b], Xun Shen [c], Mingwu Zhang [d]

[a] Graduate School of Informatics and Engineering, The University of Electro-Communications, Tokyo, Japan
[b] Department of Informatics, Kyushu University, Fukuoka, Japan
[c] Graduate School of Engineering, Tokyo University of Agriculture and Technology, Tokyo, Japan
[d] School of Computers, Hubei University of Technology, Wuhan, China

ARTICLE INFO

Keywords:
Federated learning
Privacy preserving
LLM security
Post-quantum cryptography
Multi-key homomorphic encryption
Secret sharing

ABSTRACT

Large Language Models (LLMs) and other foundation models are increasingly trained and fine-tuned using data distributed across organizations and edge devices. Federated Learning (FL) is a natural paradigm for such collaboration, yet privacy leakage from model updates remains a major barrier, especially because powerful generative priors can amplify reconstruction and inference attacks in the LLM era. Privacy-Preserving Federated Learning (PPFL) based on multi-key homomorphic encryption, notably the CDKS scheme proposed by Chen et al. at CCS 2019, enables server-side aggregation over encrypted updates without sacrificing model utility. However, CDKS-based PPFL requires each participant to disclose a partial decryption value, and naive disclosure allows an honest-but-curious server (or colluding participants) to recover individual plaintext updates.

In this paper, we propose *CDKS-LSS*, a PPFL framework that integrates Shamir-style Linear Secret Sharing (LSS) into the CDKS decryption workflow. Our design (i) removes the need for a trusted third party, (ii) upgrades a critical part of the decryption-stage confidentiality from computational security to information-theoretic security via LSS over R_q, and (iii) supports latency-tolerant aggregation by allowing the server to complete decryption after receiving only a threshold t of shares rather than all N participants. We formalize security under a semi-honest and collusion model and analyze communication costs. The resulting protocol provides a practical, post-quantum-secure cryptographic primitive for privacy-preserving federated training and fine-tuning of LLMs and other high-capacity models in cross-silo and cross-device settings.

1. Introduction

In recent years, Large Language Models (LLMs) and other foundation models have rapidly become core components of modern information systems. Their strong generalization capability is powered by large-scale training and continual fine-tuning, which increasingly relies on data distributed across organizations (e.g., healthcare consortia, financial institutions, and public agencies) and edge devices (e.g., smartphones and IoT). While such data is highly valuable for improving model quality and robustness, centralizing it is often infeasible due to privacy regulations, business confidentiality, and security risks. These challenges motivate collaborative learning paradigms that preserve data locality.

* Corresponding author.
Email addresses: y-wang@uec.ac.jp (Y. Wang), i2630012@gl.cc.uec.ac.jp (F. Inoue).

https://doi.org/10.1016/j.ins.2026.123619
Received 12 February 2026; Received in revised form 8 May 2026; Accepted 9 May 2026
Available online 12 May 2026
0020-0255/© 2026 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Federated Learning (FL), including federated fine-tuning of LLMs/foundation models, enables multiple participants to train a shared model without directly exchanging raw data: each participant performs local training and only communicates model updates to a server for aggregation. Despite this appealing workflow, privacy protection remains a critical obstacle to practical deployment. It is well known that local updates can leak sensitive information through reconstruction or inference attacks, and the risk is arguably amplified in the LLM era because powerful generative priors can improve an adversary's ability to recover semantically meaningful content from gradients or updates. Therefore, establishing *Privacy-Preserving Federated Learning* (PPFL) mechanisms that keep local updates confidential is essential for next-generation LLM-centric applications.

Two mainstream approaches to PPFL are differential privacy and cryptographic secure aggregation. Differential privacy can provide formal privacy guarantees by injecting noise, but it often introduces a non-trivial utility–privacy trade-off, which can be particularly undesirable for high-capacity models and costly fine-tuning workloads. Cryptographic approaches based on homomorphic encryption enable computation over encrypted updates and therefore avoid sacrificing accuracy due to noise injection. In particular, *Multi-Key Homomorphic Encryption* (MKHE) is attractive for PPFL because each participant uses its own key pair while still supporting aggregation across ciphertexts encrypted under different keys.

This work focuses on a representative lattice-based cryptographic scheme called CDKS [3]. The security of CDKS is based on the hardness of the *Ring Learning with Errors* (RLWE) problem. Namely, besides offering strong computational security grounded in lattice assumptions, RLWE-based constructions are natural candidates for deployment in the post-quantum era [4], for instance RLWE-based key exchange algorithms [5] and encryption schemes [6,7]. However, when CDKS is naively applied to PPFL, the decryption stage becomes a major leakage point: each participant must provide a partial decryption value, and combining this value with publicly available ciphertext components can enable an honest-but-curious server (or other participants) to recover that participant's plaintext update. Existing countermeasures either rely on an "aggregated public key" that is fragile under participant dropouts, or introduce a *Trusted Third Party* (TTP) for key management and decryption, which is often impractical.

It is known that the development of quantum computers has been progressing rapidly recently, with practical implementation anticipated in the 2030s. Once quantum computers become practical, the security of many widely used cryptographic schemes, such as RSA encryption and elliptic curve cryptography, will be threatened. In response to this concern, as of March 2025, five schemes have been selected as standardization candidates in the PQC standardization project by the National Institute of Standards and Technology (NIST), three of which are lattice-based schemes [8]. RLWE problem with proper parameters is considered to withstand threats from classical computers and outperform quantum computing. Thus, by establishing PPFL based on CDKS, it is possible to realize a federated learning method that can be operated securely even in the post-quantum era.

In PPFL based on CDKS, each participant needs to securely share the results of partial decryption performed using their own secret key. This sharing method is one of the challenges in PPFL using CDKS, and various proposals have been made. For example, Ma et al. proposed a scheme called xMK-CKKS based on the so-called MK-CKKS (i.e., CDKS) in [1]. In xMK-CKKS, encryption is performed using an "aggregated public key" generated by adding the public keys of all participants, thereby improving the security of the decryption process. However, since the aggregated public key is generated by all participants, decryption fails if even a single participant drops out during aggregation by the server. On the other hand, Wang et al. proposed a scheme that introduces a TTP, where the TTP is responsible for all secret key management and decryption in CDKS [2]. While this proposal addresses the security issues related to sharing the decryption process in CDKS, it faces the challenge that securing a TTP is difficult in practice.

The objective of this study is to eliminate the risk of privacy information leakage in the decryption process of PPFL using CDKS. To achieve this objective, this study proposes a new PPFL method that applies Shamir's *linear secret sharing* (LSS) [9] to the decryption process, thereby realizing privacy protection while overcoming the problems of existing methods.

1.1. Our contribution

We propose a new PPFL framework, termed *CDKS-LSS*, that eliminates the above decryption-stage leakage without requiring a TTP. The key idea is to incorporate Shamir-style LSS into the CDKS decryption workflow (instantiated over R_q using an exceptional sequence of evaluation points). This modification yields following three concrete benefits.

1. *TTP-free* decryption without revealing individual partial decryptions;
2. strengthening a critical part of the protocol from computational security to *information-theoretic security* via LSS;
3. *latency-tolerant* aggregation, allowing the server to complete decryption after receiving only a threshold t of shares rather than all N participants, which better matches realistic FL deployments.

A comparison between this study and related work is presented in Table 1.

Furthermore, beyond standard FL workloads, CDKS-LSS provides a practical cryptographic primitive for privacy-preserving federated training and fine-tuning of LLMs and other foundation models, where (i) confidentiality of model updates is crucial, (ii) participant availability can be intermittent, and (iii) post-quantum resilience is increasingly relevant for long-lived sensitive data.

1.2. Organization

Section 2 reviews preliminaries on federated averaging, the CDKS MKHE scheme, and Shamir-style LSS over R_q. Section 3 surveys CDKS-based PPFL designs and clarifies the decryption-stage leakage issue. Section 4 presents our proposal CDKS-LSS and the resulting PPFL protocol with threshold-based aggregation. Section 5 proves security under a semi-honest and collusion model, analyzes

Table 1
Comparison between this study and related work [1,2].

Method	TTP-free	Aggregation threshold	Security basis
Ma et al.	○	N	Computational security (RLWE)
Wang et al.	×	1	Computational security (RLWE)
Our proposal	○	t	Computational security (RLWE) and information-theoretic security (LSS)

communication overhead, and discusses implications for privacy-preserving federated training and fine-tuning of LLMs and other foundation models. Finally, Section 6 concludes the work.

2. Preliminary

2.1. Notation

This section summarizes the notation and preliminaries used throughout the paper.

$\mathbb{F}$	A finite field.
$x \xleftarrow{\$} S$	Sample x uniformly at random from a finite set S.
R	The cyclotomic ring $R = \mathbb{Z}[X]/(X^n + 1)$, where n is a power of two.
R_q	The residue ring modulo q, i.e., $R_q = R/qR$.
χ	The key distribution.
ψ	The error distribution.
ϕ	An error distribution with larger variance than ψ.
N	The number of clients in LLMs (i.e., participants in FL).
P_i	The i-th client/participant.
$\mathcal{D}_i$	The local dataset of P_i.
R	The total number of federated learning rounds.
r	The round index, $r \in \{1, 2, \dots, R\}$.
w_G^r	The global model weights at round r.
w_i^r	The local model weights of P_i at round r.
pk_i	The public key of P_i.
sk_i	The secret key of P_i.
ct_i	A ciphertext encrypted under pk_i.
$c_{i,0}, c_{i,1} \in R_q$	The first and second components of ct_i.
t	threshold.

2.2. Ring learning with errors (RLWE) problem [10]

Let $s \xleftarrow{\$} \chi$ be a secret polynomial, $e \xleftarrow{\$} \psi$ be an error polynomial, and $a \xleftarrow{\$} R_q$. Then, the pair $(a, b = a \cdot s + e) \in R_q^2$ is called an RLWE sample. The RLWE *search* problem with parameters (n, q, χ, ψ) is to recover the secret s given poly(n) independent samples (a_i, b_i). The RLWE *decision* problem is to distinguish the RLWE distribution from the uniform distribution over R_q^2. The RLWE problem is hard if the advantage of any probabilistic polynomial-time adversary is negligible.

2.3. Federated averaging [11]

Fig. 1 illustrates the standard Federated Averaging (FedAvg) procedure. We describe the workflow at round r. First, each participant downloads the previous-round global model weights w_G^{r-1} from the server. Next, participant P_i performs local training using w_G^{r-1} and its local dataset $\mathcal{D}_i$, thereby updating the local model and obtaining w_i^r. After receiving $\{w_i^r\}_{i=1}^N$ from the participants, the server aggregates them to update the global model by computing $w_G^r := \frac{1}{N}\sum_{i=1}^N w_i^r$. This procedure is repeated until a predefined accuracy criterion is satisfied.

2.4. Chen–Dai–Kim–Song (CDKS) multi-key homomorphic encryption [3]

CDKS is a multi-key homomorphic encryption scheme that enables computations over data encrypted under distinct public keys pk_i of multiple participants P_i $(1 \le i \le N)$. Its security relies on the hardness of the RLWE problem. We summarize the CDKS procedures used in this paper below. Since our scheme does not require homomorphic multiplication, we describe only homomorphic addition.

- $\mathsf{CDKS.Setup}(1^\lambda)$**:** Given the security parameter λ, set parameters n, q, χ, ψ. Sample a uniformly random polynomial $a \xleftarrow{\$} R_q$ as a common reference string, and output the public parameters$pp = (n, q, \chi, \psi, a)$.

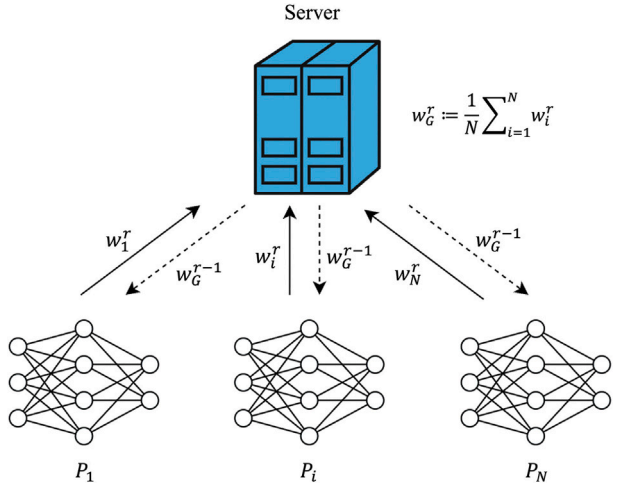


Fig. 1. Federated Averaging (FedAvg).

- CDKS.KeyGen(pp)**:** Participant P_i samples a secret $s_i \xleftarrow{\$} \chi$ and an error $e_i \xleftarrow{\$} \psi$, and generates $\mathsf{sk}_i = (1, s_i)$ and $\mathsf{pk}_i = (b_i, a)$ with

$$b_i = -a \cdot s_i + e_i \in R_q.$$

- CDKS.Enc(m_i, pk_i)**:** Participant P_i samples $v_i \xleftarrow{\$} \chi$ and $e_{i,0}, e_{i,1} \xleftarrow{\$} \psi$, and outputs a ciphertext $ct_i = (c_{i,0}, c_{i,1}) \in R_q^2$ encrypting $m_i \in R$ under pk_i as

$$c_{i,0} = v_i \cdot b_i + m_i + e_{i,0} \pmod q,$$
$$c_{i,1} = v_i \cdot a + e_{i,1} \pmod q.$$

- CDKS.Add($ct_1, \ldots, ct_N$)**:** Given N ciphertexts $\{ct_i = (c_{i,0}, c_{i,1})\}_{i=1}^N$, an aggregated ciphertext $\overline{ct}_{\text{add}} = (\overline{c}_0, \overline{c}_1, \ldots, \overline{c}_N) \in R_q^{N+1}$ is formed so as to retain all second components. Specifically,

$$\overline{c}_0 = \sum_{i=1}^{N} c_{i,0} \pmod q,$$
$$\overline{c}_i = c_{i,1} \pmod q \ (1 \le i \le N),$$

and hence

$$\overline{ct}_{\text{add}} = (\overline{c}_0, \overline{c}_1, \ldots, \overline{c}_N) \in R_q^{N+1}.$$

- CDKS.Dec($\overline{ct}_{\mathbf{add}}, \{\mathsf{sk}_i\}_{i=1}^N$)**:** Decryption proceeds in two steps:

1. **Partial Decryption:** Each participant P_i samples $e_i^* \xleftarrow{\$} \phi$ and computes

$$\mu_i = \overline{c}_i \cdot s_i + e_i^* \pmod q.$$

2. **Merge:** The server merges all shares and outputs

$$\mu = \overline{c}_0 + \sum_{i=1}^{N} \mu_i \pmod q.$$

The merged value μ approximates the plaintext sum $M = \sum_{i=1}^N m_i$, i.e., $\mu \approx M$.

Correctness.

$$\begin{aligned}\mu &= \overline{c}_0 + \sum_{i=1}^{N} \mu_i \pmod q \\ &= \overline{c}_0 + \sum_{i=1}^{N} (\overline{c}_i \cdot s_i + e_i^*) \pmod q \\ &= \left(\sum_{i=1}^{N} c_{i,0}\right) + \sum_{i=1}^{N} (c_{i,1} \cdot s_i) + \sum_{i=1}^{N} e_i^* \pmod q\end{aligned}$$

$$= \sum_{i=1}^{N}(c_{i,0} + c_{i,1} \cdot s_i) + \sum_{i=1}^{N} e_i^* \pmod q.$$

For each i, using $b_i = -a \cdot s_i + e_i$, we have

$$\begin{aligned} c_{i,0} + c_{i,1} \cdot s_i \pmod q &= (v_i \cdot b_i + m_i + e_{i,0}) \\ &\quad + (v_i \cdot a + e_{i,1}) \cdot s_i \pmod q \\ &= m_i + v_i \cdot e_i + e_{i,0} + e_{i,1} \cdot s_i \pmod q. \end{aligned}$$

Therefore,

$$\begin{aligned} \mu &= \sum_{i=1}^{N}(m_i + v_i \cdot e_i + e_{i,0} + e_{i,1} \cdot s_i) + \sum_{i=1}^{N} e_i^* \pmod q \\ &= M + \sum_{i=1}^{N} v_i \cdot e_i + \sum_{i=1}^{N} e_{i,0} + \sum_{i=1}^{N} e_{i,1} \cdot s_i \\ &\quad + \sum_{i=1}^{N} e_i^* \pmod q \\ &\approx M \pmod q. \end{aligned}$$

Vulnerability of federated learning based on CDKS.

A serious vulnerability arises when CDKS is naively applied to federated learning. Specifically, a server or other participants can recover a target participant's plaintext m_i using only publicly available information. We describe this recovery procedure below.

In CDKS.Enc, participant P_i generates a ciphertext ct_i, whose component $c_{i,0}$ is computed using the public key $b_i \approx -a \cdot s_i$ as

$$c_{i,0} = v_i \cdot b_i + m_i + e_0 \approx m_i - v_i \cdot a \cdot s_i \pmod q.$$

On the other hand, in the partial decryption phase of CDKS.Dec, participant P_i computes and shares μ_i with the server or other participants. Since the ciphertext component satisfies $c_{i,1} \approx v_i \cdot a$, μ_i can be expressed as

$$\mu_i = c_{i,1} \cdot s_i + e_i^* \approx v_i \cdot a \cdot s_i \pmod q.$$

An adversary can then estimate the plaintext m_i by adding the publicly available $c_{i,0}$ and the revealed μ_i:

$$c_{i,0} + \mu_i \approx (m_i - v_i \cdot a \cdot s_i) + (v_i \cdot a \cdot s_i) = m_i \pmod q.$$

Therefore, when CDKS is directly employed in federated learning, participants' private information is not protected.

2.5. Shamir's linear secret sharing [9]

Shamir's Linear Secret Sharing (LSS) realizes a (t, N) threshold scheme based on polynomial interpolation over a finite field. We describe the procedure for sharing an arbitrary secret $s \in \mathbb{F}$ with threshold t.

- Shamir.Setup First, fix the sharing space as $\mathbb{F}$ and assign each participant P_i a distinct nonzero public evaluation point $\alpha_i \in \mathbb{F}$.
- Shamir.Share$(s, t, \{\alpha_i\}_{i=1}^N)$**:** The dealer samples a random polynomial $f(X)$ over $\mathbb{F}$ of degree $t-1$ whose constant term is s:

$$f(X) = s + \sum_{i=1}^{t-1} \lambda_i X^i, \quad \lambda_i \xleftarrow{\$} \mathbb{F}.$$

  It then computes and distributes to participant P_i the share$s_i = f(\alpha_i) \in \mathbb{F}$.
- Shamir.Combine$(\{s_j\}_{j\in T}, \{\alpha_j\}_{j\in T})$**:**

  Let $T \subseteq \{1, \dots, N\}$ be an index set with $|T| \geq t$. Using Lagrange interpolation, the secret s is reconstructed from the shares held by participants in T as

$$s = \sum_{j\in T} s_j \cdot \prod_{k\in T, k\neq j} \frac{\alpha_k}{\alpha_k - \alpha_j} \in \mathbb{F}.$$

2.6. Shamir's linear secret sharing over R_q [12,13]

In contrast to LSS over a finite field $\mathbb{F}$, LSS over the ring R requires the public evaluation points to form an Exceptional Sequence, defined as follows.

Definition 1 (Exceptional Sequence). Let $\alpha_1, \dots, \alpha_N \in R$ be a sequence of elements. If, for every $i \neq j$, the difference $\alpha_i - \alpha_j$ is a unit in R, then the sequence is called an Exceptional Sequence.

When the public evaluation points satisfy the Exceptional Sequence condition, the following theorem holds.

Theorem 1 (**Correctness of Shamir's Linear Secret Sharing Constructed over** R). *Let R be a commutative ring, and let $\alpha_1, \ldots, \alpha_N$ be an Exceptional Sequence in R. Then, Shamir's linear secret sharing scheme constructed over R using the public evaluation points $\alpha_1, \ldots, \alpha_N$ is correct and secure.*

By Theorem 1, if the public evaluation points form an Exceptional Sequence in R_q, then LSS can also be instantiated over R_q.

2.7. Share re-sharing over R_q [13]

Based on LSS over R_q, we describe a share re-sharing protocol that reconstructs only the sum $S = \sum_{i=1}^{N} s_i$ under a (t, N) threshold, while keeping each individual secret $s_i \in R_q$ hidden.

- **Preparation:** Fix the sharing space as R_q and assign each participant P_i a public evaluation point $\alpha_i \in R_q$ such that $\{\alpha_i\}_{i=1}^{N}$ satisfies the Exceptional Sequence condition.
- **Share generation and distribution:** For its secret s_i, each participant P_i generates a random polynomial $f_i(X)$ over R_q of degree $t-1$ following the procedure of Shamir.Share. Then, for each participant P_j $(1 \le j \le N)$, P_i computes a share $s_{i,j}$ and sends it to P_j, where

$$s_{i,j} = f_i(\alpha_j) \in R_q.$$

- **Share aggregation:** Each participant P_i sums the received shares and defines the result as its aggregated share $\tilde{s}_i$:

$$\tilde{s}_i = \sum_{j=1}^{N} s_{j,i} = \sum_{j=1}^{N} f_j(\alpha_i) \in R_q.$$

  By linearity, $\tilde{s}_i$ is equal to $F(\alpha_i)$ for the combined polynomial $F(X) = \sum_{j=1}^{N} f_j(X)$. Since the constant term of $F(X)$ satisfies $F(0) = \sum_{j=1}^{N} s_j = S$, the set $\{\tilde{s}_i\}_{i=1}^{N}$ forms valid shares of the sum S.
- **Reconstruction of the sum:** Any subset T of participants with $|T| \ge t$ pools their aggregated shares $\{\tilde{s}_i\}_{i \in T}$ and runs Shamir.Combine. This reconstructs only the sum S while keeping the individual values s_i hidden:

$$S = \sum_{i \in T} \tilde{s}_i \cdot \prod_{k \in T, k \neq i} \frac{\alpha_k}{\alpha_k - \alpha_i} \in R_q.$$

3. Related works

In this section, we review two federated learning schemes based on CDKS: the xMK-CKKS-based approach proposed by [1] and the MKHA-PPFL scheme proposed by [2].

3.1. LLM/FMs, FL privacy, and LLM-era threats

Large Language Models (LLMs) and other foundation models are increasingly trained or fine-tuned using distributed data, making Federated Learning (FL) a natural candidate for cross-silo collaboration under privacy constraints. However, privacy risks in such settings remain substantial. Recent literature highlights that fine-tuning LLMs can be vulnerable to privacy attacks such as membership inference and data extraction, motivating stronger protection of training signals and update information [14,15]. Moreover, the attack surface in FL is evolving: gradient/update inversion can be significantly strengthened when the adversary leverages powerful generative priors, and more recently, language- or vision-language foundation models have been shown to further enhance gradient inversion attacks by enabling targeted reconstructions guided by natural-language descriptions [16,17]. These results reinforce the need for cryptographic PPFL mechanisms (and, more broadly, federated fine-tuning of foundation models) whose confidentiality does not rely on assuming a weak adversary or limited prior knowledge. In addition, practical deployments of foundation-model fine-tuning frequently face stragglers and client dropouts, which motivate threshold-based aggregation and decryption designs [18].

3.2. xMK-CKKS: a multi-key homomorphic encryption scheme and its application to PPFL

xMK-CKKS [1] extends MK-CKKS, together with a PPFL framework built upon it. Note that MK-CKKS and CDKS refer to the same cryptographic scheme. In xMK-CKKS, encryption is performed under an *aggregated public key* obtained by summing all participants' public keys, thereby eliminating a weakness in the decryption process.

3.2.1. xMK-CKKS

- xMK-CKKS.Setup(1^{λ})**:** This algorithm is identical to CDKS.Setup. Given the security parameter λ, it outputs the public parameters as$pp = (n, q, \chi, \psi, a)$.

- xMK-CKKS.KeyGen(pp)**:** Each participant P_i samples a secret key $s_i \xleftarrow{\$} \chi$ and an error term $e_i \xleftarrow{\$} \psi$, and computes an individual public key $b_i = -a \cdot s_i + e_i \in R_q$. Then, the public keys of all participants are summed to generate an aggregated public key $\tilde{b}$:

$$\tilde{b} = \sum_{i=1}^{N} b_i = -a \cdot \left(\sum_{i=1}^{N} s_i \right) + \sum_{i=1}^{N} e_i \pmod{q}.$$

  The value $\tilde{b}$ behaves as if it were a public key corresponding to a single secret key $s_{\text{add}} = \sum_{i=1}^{N} s_i$.

- xMK-CKKS.Enc($m_i, \tilde{b}$)**:** Participant P_i encrypts the plaintext m_i using the aggregated public key $\tilde{b}$. Using randomness $v_i \xleftarrow{\$} \chi$ and errors $e_{i,0}, e_{i,1} \xleftarrow{\$} \psi$, the ciphertext $ct_i = (c_{i,0}, c_{i,1})$ is generated as follows:

$$c_{i,0} = v_i \cdot \tilde{b} + m_i + e_{i,0} \pmod{q},$$
$$c_{i,1} = v_i \cdot a + e_{i,1} \pmod{q}.$$

- xMK-CKKS.Add($ct_1, \ldots, ct_N$)**:** The sum of the N ciphertexts is computed to obtain an aggregated ciphertext $\overline{ct}_{\text{add}} = (\bar{c}_0, \bar{c}_1)$:

$$\begin{aligned} \overline{ct}_{\text{add}} &= \sum_{i=1}^{N} ct_i = (\bar{c}_0, \bar{c}_1) \pmod{q} \\ &= \left(\sum_{i=1}^{N} c_{i,0}, \sum_{i=1}^{N} c_{i,1} \right) \pmod{q}. \end{aligned}$$

- xMK-CKKS.Dec($\overline{ct}_{\mathbf{add}}, \{s_i\}_{i=1}^{N}$)**:** Decryption is performed as follows:

1. **Partial Decryption:** Each participant P_i computes D_i using $\bar{c}_1$ and its secret key s_i, adding $e_i^* \xleftarrow{\$} \phi$:

$$D_i = \bar{c}_1 \cdot s_i + e_i^* \pmod{q}.$$

2. **Merge:** The plaintext sum $M = \sum_{i=1}^{N} m_i$ is recovered from all participants' D_i as

$$M \approx \bar{c}_0 + \sum_{i=1}^{N} D_i \pmod{q}.$$

Correctness.

$$\begin{aligned} \bar{c}_0 + \sum_{i=1}^{N} D_i &= \sum_{i=1}^{N} c_{i,0} + \sum_{i=1}^{N} c_{i,1} \cdot s_i + \sum_{i=1}^{N} e_i^* \pmod{q} \\ &= \sum_{i=1}^{N} (v_i \cdot \tilde{b} + m_i + e_{i,0}) + \sum_{i=1}^{N} (v_i \cdot a + e_{i,1}) \cdot \sum_{i=1}^{N} s_i + \sum_{i=1}^{N} e_i^* \pmod{q} \\ &= \sum_{i=1}^{N} m_i + \sum_{i=1}^{N} v_i \sum_{i=1}^{N} e_i + \sum_{i=1}^{N} s_i \cdot e_{i,1} + \sum_{i=1}^{N} e_i^* \pmod{q} \\ &\approx M \pmod{q}. \end{aligned}$$

3.2.2. PPFL based on xMK-CKKS

One training round of PPFL based on xMK-CKKS consists of the following steps. An overview of the xMK-CKKS-based PPFL protocol is illustrated in Fig. 2.

Setup: Each participant P_i runs CDKS.KeyGen(pp) to generate a public key b_i and a secret key s_i. The participants then share their public keys and compute the aggregated public key $\tilde{b} = \sum_{i=1}^{N} b_i$.

1. **Local training and encryption:** Each participant P_i performs local training using the previous-round global model weights w_G^{r-1} and its local dataset D_i, and obtains the local update w_i^r. The participant then encrypts w_i^r under the aggregated public key $\tilde{b}$ and sends the resulting ciphertext ct_i to the server:

$$ct_i = \text{xMK-CKKS.Enc}(w_i^r, \tilde{b}).$$

2. **Server-side aggregation:** The server collects the $\{ct_i\}_{i=1}^{N}$ from all participants and generates an aggregated ciphertext $\overline{ct}_{\text{add}}$ by running xMK-CKKS.Add:

$$\overline{ct}_{\text{add}} = \text{xMK-CKKS.Add}(ct_1, \ldots, ct_N).$$

The server then sends the component $\bar{c}_1$ of the aggregated ciphertext to all participants.

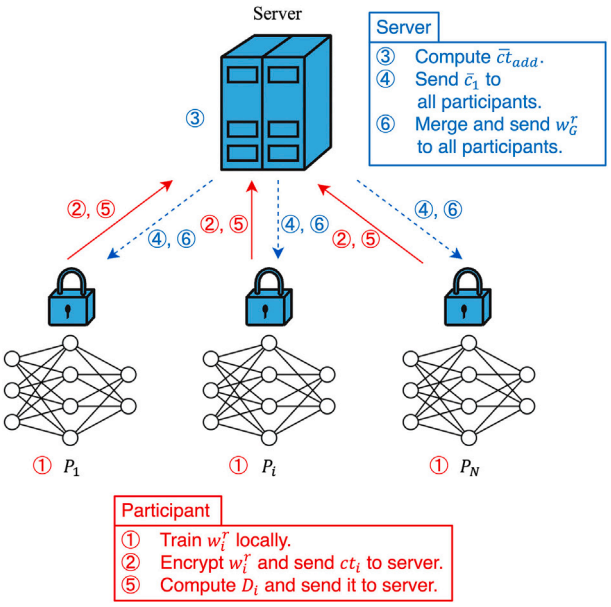


Fig. 2. PPFL based on xMK-CKKS.

3. **Decryption and update:** Each participant P_i computes D_i using the received $\bar{c}_1$ and its secret key s_i, and sends D_i to the server. The server aggregates the shares $\{D_i\}_{i=1}^N$ and performs Merge to recover the sum of all participants' updates $w_{\text{add}} = \sum_{i=1}^N w_i^r$. The server then averages the sum (i.e., $w_G^r = \frac{1}{N} w_{\text{add}}$) and distributes the updated global model weights w_G^r to the participants.

In xMK-CKKS-based PPFL, the aggregated public key is constructed jointly by all participants; hence, if even a single participant drops out during a round, decryption fails. Moreover, the server cannot execute the Merge step until it has collected all D_i from the N participants.

3.2.3. Security

Ma et al. establish the security of their proposed scheme through the following theorems.

Theorem 2 (**Security against a semi-honest server**). *A semi-honest server cannot infer participants' private information from the information obtained during protocol execution.*

Theorem 3 (**Security against semi-honest participants**). *Even if semi-honest participants eavesdrop on information shared by other participants, they cannot infer those participants' private information.*

Theorem 4 (**Security against collusion between the server and multiple participants**). *Even if the server colludes with $k < N-1$ participants, no information about the model updates of the remaining participants is disclosed, where N denotes the total number of participants and k denotes the number of colluding participants.*

3.3. PPFL with a trusted third party (TTP)

In the federated learning scheme MKHA-PPFL proposed by [2], it targets confidential aggregation in settings including LLM/foundation-model fine-tuning. There is a Trusted Third Party (TTP) incorporated alongside the participants and the server to manage keys and perform decryption.

We describe the workflow of MKHA-PPFL. An overview of MKHA-PPFL is shown in Fig. 3. One training round of MKHA-PPFL consists of the following steps, where n_w denotes the number of participants in the round.

Setup: $\mathsf{CDKS.Setup}(1^\lambda)$ and $\mathsf{CDKS.KeyGen}(pp)$ are executed by the TTP, which then distributes an individual public key pk_i to each participant P_i. The corresponding secret key sk_i is securely stored and managed by the TTP.

1. **Local training and encryption:** Each participant P_i performs local training using the previous-round global model weights w_G^{r-1} and its local dataset D_i, and computes a local update w_i^r. The participant then encrypts w_i^r under its public key and sends the ciphertext ct_i to the server:

$$ct_i = \mathsf{CDKS.Enc}(w_i^r, \mathsf{pk}_i).$$

2. **Server-side aggregation:** The server collects ciphertexts $\{ct_i\}_{i=1}^{n_w}$ from the n_w participants. It aggregates them using CDKS.Add and applies averaging in the encrypted domain, thereby obtaining a ciphertext $\bar{ct}_G$ corresponding to the updated global model.

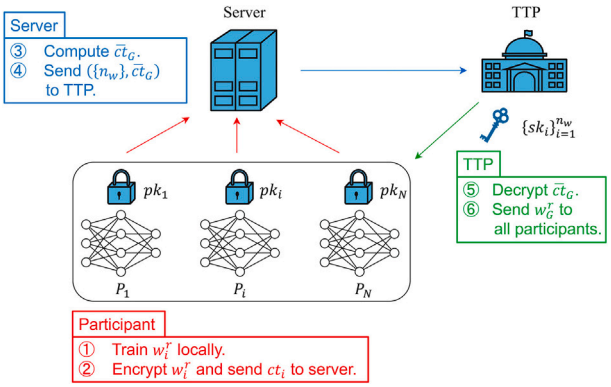


Fig. 3. MKHA-PPFL.

The server sends $\overline{ct}_G$ to the TTP together with the index set of the participants involved in the round:

$$\overline{ct}_G = \frac{1}{n_w}\mathsf{CDKS.Add}(ct_1, \ldots, ct_{n_w}).$$

3. **Decryption and update by the TTP:** The TTP decrypts $\overline{ct}_G$ using the set of secret keys $\{\mathsf{sk}_i\}_{i=1}^{n_w}$ corresponding to the participants in the round, and distributes the resulting global model weights w_G^r to the participants.

In MKHA-PPFL, delegating secret-key management and decryption to the TTP strengthens security: even in the presence of malicious attempts to infer other participants' updates, neither participants nor the server can access one another's model-update information. Moreover, since the TTP performs the entire decryption procedure, the global model can be updated immediately once a sufficient number of ciphertexts have been aggregated. However, deploying a fully trusted TTP is often impractical in real-world settings.

4. Our proposal: CDKS-LSS

4.1. Threat model

In this study, we assume that all entities involved, including the server, are *semi-honest*. That is, each participant follows the protocol specification faithfully while attempting to infer other participants' private information from the data shared during protocol execution. In addition, we consider the possibility that the server may collude with a subset of participants and jointly mount attacks by pooling their information.

4.2. CDKS-LSS

We propose a new scheme, termed **CDKS-LSS**, which incorporates Linear Secret Sharing (LSS) into the decryption procedure of CDKS. CDKS-LSS follows CDKS for the setup, key generation, encryption, and homomorphic addition phases. We present the decryption procedure of CDKS-LSS, denoted by CDKS.Dec-LSS, in Algorithm 1, and outline its main steps below.

Conditions: The public evaluation points are chosen so as to satisfy the *Exceptional Sequence* condition, according to Definition 1 and Theorem 1. We also assume $t \leq N$.

1. **Partial decryption and re-sharing:** Each participant P_i computes a partial decryption value $\mu_i = c_{i,1} \cdot s_i \in R_q$ using its secret key s_i and $c_{i,1}$. Then, P_i randomly samples a degree-$(t-1)$ polynomial $f_i(X)$ whose constant term is μ_i. For example, for participant P_j, P_i evaluates f_i at the public evaluation point α_j to obtain $\tilde{\mu}_{i,j} = f_i(\alpha_j)$, and sends it to P_j.
2. **Share aggregation:** Participant P_i computes the sum of received shares $\tilde{\mu}_i = \sum_{j=1}^{N} \tilde{\mu}_{j,i}$.
3. **Decryption:** Each participant sends the aggregated share $\tilde{\mu}_i$ to the server. The server computes μ_i' only for the first t shares received (in arrival order), and reconstructs the final decryption result as $\mu = \overline{c}_0 + \sum_{i=1}^{t} \mu_i'$.

4.3. CDKS-LSS-based PPFL

Fig. 4 illustrates an overview of the federated learning framework built on CDKS-LSS. In addition, the detailed procedure is provided in Algorithm 2.

We summarize Algorithm 2 as follows. First, each participant P_i performs local training using the global model weights from the previous round, w_G^{r-1}, and its local dataset D_i, and computes a local update w_i^r. Next, P_i encrypts w_i^r under its public key, and sends the ciphertext component $c_{i,0}$ of ct_i to the server. In Lines 8–15, each participant proceeds according to CDKS.Dec-LSS to generate $\tilde{\mu}_i$ and transmits it to the server. On the server side, it first aggregates the received $c_{i,0}$ values to compute $\overline{c}_0 = \sum_{i=1}^{N} c_{i,0}$. Moreover, in

Algorithm 1: CDKS.Dec-LSS.

Setting:
- public evaluation points: $(\alpha_1, \ldots, \alpha_N)$
- threshold: t

Input: $ct_i \in R_q^2, s_i \in \chi$
Output: $\mu = \bar{c}_0 + \sum_{i=1}^{t} \mu_i' \in R_q$

1 **for** each participant P_i **do**
2 **Send** $c_{i,0}$ to **Server**
3 $\mu_i \leftarrow c_{i,1} \cdot s_i$
4 $\lambda_{i,1}, \ldots, \lambda_{i,t-1} \leftarrow R_q$
5 **for** $j = 1, 2, \ldots, N$ **do**
6 $\tilde{\mu}_{i,j} \leftarrow f_i(\alpha_j) = \mu_i + \sum_{k=1}^{t-1} \lambda_{i,k}\, \alpha_j^k$
7 Send $\tilde{\mu}_{i,j}$ to participant P_j $(j \neq i)$
8 $\tilde{\mu}_i \leftarrow \sum_{j=1}^{N} \tilde{\mu}_{j,i}$
9 **Send** $\tilde{\mu}_i$ to **Server**
10 **Server operations:**
11 $\bar{c}_0 \leftarrow \sum_{i=1}^{N} c_{i,0}$
12 $\mathcal{P} := \{\, i \in \{1, \ldots, N\} \mid \tilde{\mu}_i$ is first received by the server $\},\quad |\mathcal{P}| = t.$
13 **for** $i \in \mathcal{P}$ **do**
14 $\mu_i' \leftarrow \tilde{\mu}_i \prod_{\substack{k=1 \\ k \neq i}}^{t} \frac{\alpha_k}{\alpha_k - \alpha_i}$
15 $\mu \leftarrow \bar{c}_0 + \sum_{i=1}^{t} \mu_i'$
16 Send μ to all participants

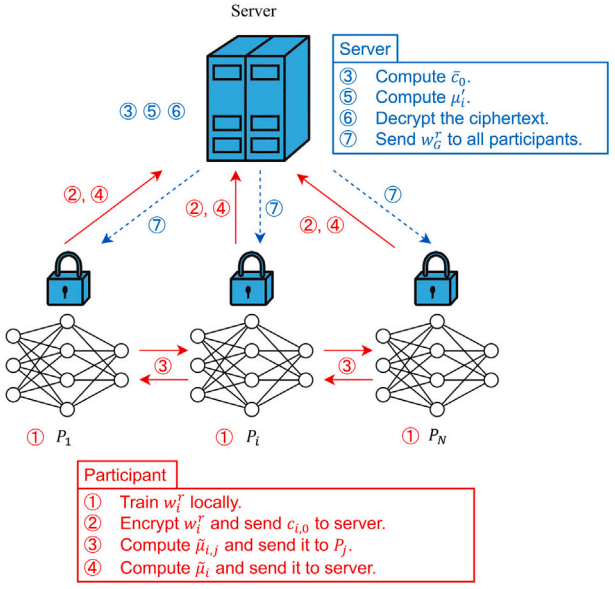


Fig. 4. PPFL based on CDKS-LSS.

Lines 18–21, the server computes the corresponding μ_i' in the order in which it receives $\tilde{\mu}_i$, and once t values of μ_i' have been collected, it performs decryption to obtain w_{add}^r. Finally, the server applies averaging $w_G^r = \frac{1}{N} w_{\text{add}}^r$ and broadcasts w_G^r to all participants.

In CDKS-LSS-Based PPFL, during the server-side aggregation, the global weights can be recovered as soon as t shares are received and the computation is completed, without waiting for all N shares to arrive. This makes the scheme more robust to participant dropouts than xMK-CKKS and enables timely updates.

Algorithm 2: PPFL Based on CDKS-LSS.

Setting:

- public parameter(pp): (n, q, χ, ψ, a)
- public evaluation points: $(\alpha_1, \ldots, \alpha_N)$
- threshold: t

Input: Local dataset $\mathcal{D}_i$
Output: Global weights w_G^R

for $r = 1, 2, \ldots, R$ **do**
  for each participant P_i **do**
    if $r = 1$ **then**
      $(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{CDKS.KeyGen}(pp)$
    $w_i^r \leftarrow \mathrm{Train}(w_G^{r-1}, \mathcal{D}_i)$
    $ct_i \leftarrow \mathsf{CDKS.Enc}(w_i^r, pk_i)$
    Send $c_{i,0}$ to **Server**
    $\mu_i \leftarrow c_{i,1} \cdot s_i$
    $\lambda_{i,1}, \ldots, \lambda_{i,t-1} \leftarrow R_q$
    for $j = 1, 2, \ldots, N$ **do**
      $\tilde{\mu}_{i,j} \leftarrow f_i(\alpha_j) = \mu_i + \sum_{k=1}^{t-1} \lambda_{i,k}\, \alpha_j^k$
      if $j \neq i$ **then**
        Send $\tilde{\mu}_{i,j}$ to participant P_j
    $\tilde{\mu}_i \leftarrow \sum_{j=1}^{N} \tilde{\mu}_{j,i}$
    Send $\tilde{\mu}_i$ to **Server**
  Server operations:
  $\bar{c}_0 \leftarrow \sum_{i=1}^{N} c_{i,0}$
  $\mathcal{P} := \{\, i \in \{1, \ldots, N\} \mid \tilde{\mu}_i \text{ is first received by the server} \,\}, \quad |\mathcal{P}| = t.$
  for $i \in \mathcal{P}$ **do**
    $\mu_i' \leftarrow \tilde{\mu}_i \prod_{\substack{i \in \mathcal{P} \\ i \neq i}}^{t} \frac{\alpha_k}{\alpha_k - \alpha_i}$
  $w_{\text{add}}^r \leftarrow \bar{c}_0 + \sum_{i \in \mathcal{P}} \mu_i'$
  $w_G^r \leftarrow \frac{1}{N} w_{\text{add}}^r$
  Send w_G^r to all participants

5. Evaluation

5.1. Security

The security of the proposed federated learning scheme is established by the following three theorems.

Theorem 5 (**Security against a semi-honest server**). *Let S be a semi-honest server, and let*

$$\mathsf{View}_S = \left(pp, \{c_{i,0}\}_{i=1}^N, \{\tilde{\mu}_i\}_{i=1}^t, M\right)$$

denote the view of S in one execution of the proposed protocol. In this view, pp is the public parameter, $c_{i,0}$ is the ciphertext component received from participant P_i, t is the threshold, $\{\tilde{\mu}_i\}_{i=1}^t$ is the set of aggregated shares received by S in the CDKS.Dec-LSS *protocol, where $\tilde{\mu}_i = \sum_{j=1}^N \tilde{\mu}_{j,i}$ for each $i = 1, \ldots, t$, and $M = \sum_{i=1}^N m_i$ is the intended aggregate output decrypted by S. Then, for every honest participant P_h, the semi-honest server S learns no information about the plaintext m_h or the partial decryption value μ_h.*

Proof. Fix an arbitrary honest participant P_h. We analyze separately the two types of information about P_h that are observable to the semi-honest server S during the protocol execution.

First, the semi-honest server S receives the ciphertext component $c_{h,0}$ of $ct_h = \mathsf{CDKS.Enc}(m_h)$. This part of the server's view is protected by the security of CDKS. Under the hardness of the RLWE problem, CDKS provides computational security against any probabilistic polynomial-time adversary that does not possess the corresponding secret key. As S does not possess the secret key of the honest participant P_h, the observed value $c_{h,0}$ does not enable S to recover, or obtain any non-negligible information about, the individual plaintext m_h. Therefore, the ciphertext layer computationally hides m_h from the semi-honest server S.

Next, we consider the aggregated shares observed by the semi-honest server S in the CDKS.Dec-LSS protocol. For each participant P_j, let $f_j(x) \in R_q[x]$ be the polynomial used in the LSS procedure, where $f_j(0) = \mu_j$ and $\tilde{\mu}_{j,i} = f_j(\alpha_i)$ for each public evaluation

point α_i. The semi-honest server S receives $\tilde{\mu}_i = \sum_{j=1}^{N} \tilde{\mu}_{j,i} = \sum_{j=1}^{N} f_j(\alpha_i)$. Define $F(x) = \sum_{j=1}^{N} f_j(x)$. Then, we have $\tilde{\mu}_i = F(\alpha_i)$ and $F(0) = \sum_{j=1}^{N} \mu_j$. Hence, the set $\{\tilde{\mu}_i\}_{i=1}^{t}$ observed by S is a valid R_q-based LSS sharing only of the aggregate decryption value $\sum_{j=1}^{N} \mu_j$, and not of any individual μ_h.

Because the public evaluation points satisfy the Exceptional Sequence condition, Theorem 1 guarantees that the underlying R_q-based LSS is information-theoretically secure. Therefore, from the aggregated shares $\{\tilde{\mu}_i\}_{i=1}^{t}$, the semi-honest server S can reconstruct only the aggregate value, but cannot isolate the individual contribution of the honest participant P_h, namely μ_h or the individual shares originating from P_h. In other words, the LSS layer hides each participant's individual decryption value information-theoretically, while revealing only the aggregate needed by the protocol.

Combining the above two arguments, the view of the semi-honest server S reveals neither the individual plaintext m_h nor the individual decryption value μ_h. Thus, the semi-honest server S cannot infer any honest participant's individual private information from the protocol execution. □

Theorem 6 (**Security against semi-honest participants**). *Let P_a be a semi-honest participant, and let*

$$\mathsf{View}_{P_a} = \left(pp, \{\tilde{\mu}_{j,a}\}_{j=1}^{N}\right)$$

denote the view of P_a in one execution of the proposed protocol. In this view, pp is the public parameter, and $\{\tilde{\mu}_{j,a}\}_{j=1}^{N}$ is the set of shares received by P_a from all participants in the CDKS.Dec-LSS *protocol. Then, for every honest participant P_h with $h \neq a$, the semi-honest participant P_a learns no information about the plaintext m_h or the partial decryption value μ_h.*

Proof. Fix the semi-honest participant P_a and another arbitrary honest participant P_h with $h \neq a$. We analyze the information about P_h that is observable to the semi-honest participant P_a during the protocol execution.

In the proposed scheme, ciphertexts generated by CDKS.Enc are sent only to the server and are not shared among participants. Hence, the ciphertext component $c_{h,0}$ of the honest participant P_h is not included in View_{P_a}. Therefore, with respect to privacy against other participants, the relevant information observable to P_a arises only from the CDKS.Dec-LSS protocol.

For the honest participant P_h, let $f_h(x) \in R_q[x]$ be the polynomial used in the LSS procedure, where $f_h(0) = \mu_h$ and $\tilde{\mu}_{h,i} = f_h(\alpha_i)$ for each public evaluation point α_i. In particular, the semi-honest participant P_a receives only the single share $\tilde{\mu}_{h,a} = f_h(\alpha_a)$ from P_h. Since the public evaluation points satisfy the Exceptional Sequence condition, Theorem 1 guarantees that the underlying R_q-based LSS is information-theoretically secure. Moreover, a single share is fewer than the reconstruction threshold t. Therefore, from the received share $\tilde{\mu}_{h,a}$, the semi-honest participant P_a cannot reconstruct μ_h and cannot obtain any information about it.

Finally, since the plaintext m_h of the honest participant P_h is encrypted and not directly shared with other participants, and since the share received by P_a reveals no information about μ_h, the semi-honest participant P_a cannot infer any private information about P_h from the information shared during the protocol execution. Thus, the semi-honest participant P_a cannot infer the honest participant P_h's plaintext m_h or partial decryption value μ_h from the information received during the protocol execution. □

Theorem 7 (**Security against collusion between the server and multiple participants**). *Let S be a semi-honest server, let $\mathcal{A} \subseteq \{1, \ldots, N\}$ be a set of semi-honest participants such that $|\mathcal{A}| < t$, where t is the threshold, and let*

$$\mathsf{View}_{S,\mathcal{A}} = \left(pp, \{c_{i,0}\}_{i=1}^{N}, \{\tilde{\mu}_i\}_{i=1}^{t}, \{\tilde{\mu}_{j,a}\}_{j=1,\ldots,N,\ a\in\mathcal{A}}, M\right)$$

denote the joint view of S and the participants in $\mathcal{A}$ in one execution of the proposed protocol. In this view, pp is the public parameter, $c_{i,0}$ is the ciphertext component received from participant P_i, $\{\tilde{\mu}_i\}_{i=1}^{t}$ is the set of aggregated shares received by S in the CDKS.Dec-LSS *protocol, where $\tilde{\mu}_i = \sum_{j=1}^{N} \tilde{\mu}_{j,i}$ for each $i = 1, \ldots, t$, $\{\tilde{\mu}_{j,a}\}_{j=1,\ldots,N,\ a\in\mathcal{A}}$ is the set of shares received by the participants in $\mathcal{A}$ in the same protocol, and $M = \sum_{i=1}^{N} m_i$ is the intended aggregate output decrypted by S. Then, for every honest participant P_h with $h \notin \mathcal{A}$, the semi-honest server S colluding with the semi-honest participants in $\mathcal{A}$ learns no information about the plaintext m_h or the partial decryption value μ_h.*

Proof. Fix an arbitrary honest participant P_h with $h \notin \mathcal{A}$. We analyze separately the two types of information about P_h that are observable to the semi-honest server S and the semi-honest participants in $\mathcal{A}$ during the protocol execution.

First, the semi-honest server S and the semi-honest participants in $\mathcal{A}$ observe the ciphertext component $c_{h,0}$ of $ct_h = \mathsf{CDKS.Enc}(m_h)$. This part of the joint view is protected by the security of CDKS. Under the hardness of the RLWE problem, CDKS provides computational security against any probabilistic polynomial-time adversary that does not possess the corresponding secret key. As the semi-honest server S and the semi-honest participants in $\mathcal{A}$ do not possess the secret key of the honest participant P_h, the observed value $c_{h,0}$ does not enable the semi-honest server S and the semi-honest participants in $\mathcal{A}$ to recover, or obtain any non-negligible information about, the individual plaintext m_h. Hence, the ciphertext layer computationally hides m_h from the semi-honest server S and the semi-honest participants in $\mathcal{A}$.

Next, we consider the shares observed in the CDKS.Dec-LSS protocol. For the honest participant P_h, let $f_h(x) \in R_q[x]$ be the polynomial used in the LSS procedure, where $f_h(0) = \mu_h$ and $\tilde{\mu}_{h,i} = f_h(\alpha_i)$ for each public evaluation point α_i. The semi-honest server S observes the aggregated shares $\{\tilde{\mu}_i\}_{i=1}^{t}$, while the semi-honest participants in $\mathcal{A}$ additionally observe the individual shares $\{\tilde{\mu}_{h,a}\}_{a\in\mathcal{A}}$ originating from P_h. Since the public evaluation points satisfy the Exceptional Sequence condition, Theorem 1 guarantees that the underlying R_q-based LSS is information-theoretically secure. Moreover, because $|\mathcal{A}| < t$, the semi-honest participants in $\mathcal{A}$ obtain fewer than t shares of the polynomial $f_h(x)$. Therefore, from the shares $\{\tilde{\mu}_{h,a}\}_{a\in\mathcal{A}}$, the semi-honest server S and the semi-honest participants in $\mathcal{A}$ cannot reconstruct μ_h and cannot obtain any information about it.

Table 2
Communication cost comparison between related work and the proposed scheme (20 participants) [1,2].

PPFL model	Sent by P_i (MB)		Total (MB)	Received by P_i (MB)			Total (MB)
	$P_i \to S$	$P_i \to P_j$		$P_i \leftarrow S$	$P_i \leftarrow P_j$	$P_i \leftarrow$ TTP	
xMK-CKKS	$10.409 + 5.205$	×	16	$5.205 + 0.437$	×	×	6
MKHA-PPFL	10.409	×	10	×	×	0.437	0.437
CDKS-LSS	2×5.205	98.895	109	0.437	98.895	×	99

Furthermore, by the same argument as in Theorem 5, the aggregated shares $\{\tilde{\mu}_i\}_{i=1}^{t}$ observed by the semi-honest server S form a valid R_q-based LSS sharing only of the aggregate decryption value $\sum_{j=1}^{N} \mu_j$, and do not reveal any individual μ_h. Hence, even after combining the aggregated shares observed by the semi-honest server S with the fewer-than-t individual shares available to the semi-honest participants in $\mathcal{A}$, the semi-honest server S and the semi-honest participants in $\mathcal{A}$ cannot isolate the contribution of the honest participant P_h.

Combining the above two arguments, the joint view of the semi-honest server S and the semi-honest participants in $\mathcal{A}$ reveals neither the individual plaintext m_h nor the individual partial decryption value μ_h. Thus, a semi-honest server colluding with fewer than t semi-honest participants cannot infer any remaining honest participant's individual private information from the protocol execution. □

5.2. Communication overhead

We quantify the per-round communication overhead of the proposed federated learning scheme, measured by the total amount of data transmitted and received per participant. Our communication-cost analysis follows the methodology of [2]. Specifically, we consider a federated learning setting with 20 participants on the MNIST dataset. For security, we adopt a 128-bit security level and use a representative parameter set with $n = 2^{14}$ and $\log_2 q = 318$.

In the proposed scheme, each participant first sends to the server the component $c_{i,0}$ of the ciphertext $ct_i \in R_q^2$ that encrypts its local update. According to [2], under the considered parameter setting, the size of a CDKS ciphertext encrypting a local update is 10.409 MB. Since our scheme transmits only $c_{i,0}$, the upload size is estimated to be half of this value, i.e., 5.205 MB. Next, each participant generates shares and sends them to the other participants. Because each share is an element of R_q, its size is estimated to be 5.205 MB, which is comparable to that of a ciphertext component. Each participant broadcasts this share to all other participants, resulting in a total outgoing traffic of $5.205\,\text{MB} \times 19 \approx 98.895$ MB. Similarly, each participant receives shares from the other 19 participants, so the total amount of data received is 98.895 MB. The participant then sends an R_q share to the server for aggregation, whose size is likewise 5.205 MB. Finally, the participant downloads the global model from the server; its size is 0.437 MB as reported in [2]. Consequently, the total amount of data transmitted by each participant per round is $5.205 + 98.895 + 5.205 \approx 109$ MB, while the total amount of data received per round is $98.895 + 0.437 \approx 99$ MB.

We also compute the communication cost of the related schemes under the same experimental setting and summarize the comparison with the proposed scheme in Table 2. In the table, $P_i \to P_j$ denotes the total amount of data transmitted from P_i to all other participants. As shown in Table 2, MKHA-PPFL, which employs a trusted third party (TTP), achieves the smallest communication volume in both transmission and reception. Although the proposed scheme still has room for improvement in participant-to-participant communication volume—both for sending and receiving—the upload/download latency between participants is approximately 790 ms over a 1 Gbps channel, which is considered acceptable. Regarding the communication cost of ciphertext exchange between the server and the participants, only a single ciphertext component needs to be transmitted in the proposed scheme. As a result, each upload or download takes only approximately 41 ms over a 1 Gbps channel—comparable to MKHA-PPFL and faster than xMK-CKKS.

5.3. Implications for LLM/FMs and LLM-era threats

Federated training and fine-tuning are increasingly considered for LLMs and other foundation models in cross-silo settings, where organizations wish to leverage distributed data without centralizing sensitive corpora. Recent studies also highlight that privacy risks during LLM fine-tuning include membership inference and data extraction, and that update/gradient leakage remains a practical concern [19]. Our protocol directly targets confidentiality of communicated updates by preventing the server from learning individual plaintext contributions during the decryption stage.

Moreover, the threat landscape is evolving: reconstruction attacks can be strengthened when adversaries have access to powerful generative priors, and language-guided or multimodal models can further amplify inversion quality [20,21]. This motivates cryptographic PPFL solutions whose privacy does not rely on assuming "weak" adversaries or limited priors.

Finally, availability issues (dropouts/stragglers) are common in realistic deployments [18,22]. Threshold-based decryption is therefore important for practical LLM-oriented PPFL pipelines, especially when combined with parameter-efficient fine-tuning (e.g., LoRA) that reduces communication.

6. Conclusions

In this study, we propose a novel decryption procedure for PPFL built upon CDKS through the employment of secret-sharing techniques. Specifically, the proposed approach incorporates LSS over the ring R_q into the CDKS decryption protocol, thereby enabling

secure sharing of partial decryption results without introducing a TTP. Through the adoption of LSS, the security guarantee for part of the decryption procedure is elevated from computational security to information-theoretic security. Moreover, the quantity of shares necessary for the server to perform aggregation is decreased from the complete set of participants N to a predetermined threshold t.

From the perspective of LLM-empowered security and privacy, CDKS-LSS provides a practical cryptographic primitive for privacy-preserving federated training and fine-tuning of LLMs and other foundation models, where both update confidentiality and deployment robustness are essential and post-quantum resilience is increasingly relevant. Future work includes reducing participant-to-participant communication, strengthening the security assumptions from the semi-honest setting to a model that also accounts for malicious participants, integrating protocol-level anomaly detection and compliance checking (potentially assisted by LLM-based analysis of logs and configurations), and validating the approach in federated fine-tuning workloads for high-capacity LLMs under realistic system constraints.

CRediT authorship contribution statement

Yuntao Wang: Writing – review & editing, Supervision, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Fumiya Inoue:** Writing – original draft, Investigation, Formal analysis. **Yujie Gu:** Writing – review & editing. **Xun Shen:** Writing – review & editing. **Mingwu Zhang:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was supported by JSPS KAKENHI Grant Numbers JP21K11751, JP26K02909, JP25K21805, JP24K20736, and JP24K16752, Japan. Dr. Yujie Gu also acknowledges support from Toyota Riken Scholar program. Dr. Xun Shen also acknowledges support from Research Organization of Information and Systems via Strategic Research Project (2023-SRP-06).

Data availability

No data was used for the research described in the article.

References

[1] J. Ma, S.-A. Naas, S. Sigg, X. Lyu, Privacy-preserving federated learning based on multi-key homomorphic encryption, Int. J. Intell. Syst. 37 (9) (2022) 5880–5901, https://doi.org/10.1002/int.22818
[2] Y. Wang, Y. Gu, X. Shen, Anti-collusion and latency-tolerant privacy-preserving federated learning framework via multi-key homomorphic aggregation, in: Proceedings of the 2025 IEEE Conference on Artificial Intelligence (CAI 2025), IEEE, Santa Clara, CA, USA, 2025, pp. 872–875, https://doi.org/10.1109/CAI64502.2025.00154
[3] H. Chen, W. Dai, M. Kim, Y. Song, Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference, in: L. Cavallaro, J. Kinder, X. Wang, J. Katz (Eds.), Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS 2019), ACM, London, UK, 2019, pp. 395–412, https://doi.org/10.1145/3319535.3363207
[4] C.S.D.I.T.L.N.I. of Standards T.U.D. of Commerce, Post-quantum cryptography | CSRC, 2017, https://csrc.nist.gov/projects/post-quantum-cryptography.
[5] J. Ding, X. Gao, T. Takagi, Y. Wang, One sample Ring-LWE with rounding and its application to key exchange, in: Applied Cryptography and Network Security - 17th International Conference, ACNS 2019, Bogota, Colombia, June 5–7, 2019, Proceedings, 2019, pp. 323–343.
[6] E. Alkim, L. Ducas, T. Pöppelmann, P. Schwabe, Post-quantum key exchange-a New Hope, in: USENIX Security Symposium, 2016, pp. 327–343.
[7] Y. Wang, Y. Ikematsu, T. Yasuda, Lattice-based public key cryptosystems invoking linear mapping mask, in: Provable Security - 16th International Conference, ProvSec 2022, 13600 of Lecture Notes in Computer Science, Springer, 2022, pp. 88–104.
[8] US Department of Commerce, National Institute of Standards and Technology, Post-quantum cryptography, 2020„https://csrc.nist.gov/projects/post-quantum-cryptography/ (Accessed: 3 February 2026).
[9] A. Shamir, How to share a secret, Commun. ACM 22 (11) (1979) 612–613, https://doi.org/10.1145/359168.359176
[10] V. Lyubashevsky, C. Peikert, O. Regev, On ideal lattices and learning with errors over rings, in: H. Gilbert (Ed.), Proceedings of Advances in Cryptology – EUROCRYPT 2010 (29th Annual International Conference on the Theory and Applications of Cryptographic Techniques), 6110 Lecture Notes in Computer Science, Springer, French Riviera, France, 2010, pp. 1–23, https://doi.org/10.1007/978-3-642-13190-5_1.
[11] H.B. McMahan, E. Moore, D. Ramage, S. Hampson, B. Agüera y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS 2017), 54 of Proceedings of Machine Learning Research, PMLR, Fort Lauderdale, FL, USA, 2017, pp. 1273–1282.
[12] M. Abspoel, R. Cramer, I. Damgård, D. Escudero, C. Yuan, Efficient information-theoretic secure multiparty computation over $\mathbb{Z}/p^k\mathbb{Z}$ via galois rings, in: Proceedings of the 17th International Conference on Theory of Cryptography (TCC 2019), Nuremberg, Germany, December 1–5, 2019, Proceedings, Part I, 11891 of Lecture Notes in Computer Science, Springer, Cham, 2019, pp. 471–501, https://doi.org/10.1007/978-3-030-36030-6_19.
[13] C. Mouchet, E. Bertrand, J. Hubaux, An efficient threshold access-structure for RLWE-based multiparty homomorphic encryption, J. Cryptol. 36 (2) (2023) 10, https://doi.org/10.1007/s00145-023-09452-8
[14] H. Du, et al., Privacy in fine-tuning large language models, arXiv preprint arXiv:2412.16504, 2024.
[15] W. Fu, et al., Membership inference attacks against fine-tuned large language models, in: NeurIPS, 2024.
[16] J. Jeon, J. Kim, K. Lee, S. Oh, J. Ok, Gradient inversion with generative image prior, in: NeurIPS, 2021.
[17] J. Shan, et al., Geminio: language-guided gradient inversion attacks in federated learning, in: ICCV, 2025.
[18] E. Hosseini, S. Chen, A. Khisti, Secure aggregation in federated learning using multiparty homomorphic encryption, CoRR arXiv preprint arXiv:2503.00581 (2025), https://doi.org/10.48550/ARXIV.2503.00581
[19] H. Du, S. Liu, L. Zheng, Y. Cao, A. Nakamura, L. Chen, Privacy in fine-tuning large language models: attacks, defenses, and future directions, in: X. Wu, M. Spiliopoulou, C. Wang, V. Kumar, L. Cao, Y. Wu, Y. Yao, Z. Wu (Eds.), Advances in Knowledge Discovery and Data Mining - 29th Pacific-Asia Conference on Knowledge Discovery and Data Mining, PAKDD 2025, Proceedings, Part IV, 15873 of Lecture Notes in Computer Science, Springer, 2025, pp. 326–344, https://doi.org/10.1007/978-981-96-8183-9_25.

[20] J. Jeon, J. Kim, K. Lee, S. Oh, J. Ok, Gradient inversion with generative image prior, in: M. Ranzato, A. Beygelzimer, Y.N. Dauphin, P. Liang, J.W. Vaughan (Eds.), Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6–14, 2021, Virtual, 2021, pp. 29898–29908.
[21] J. Shan, Z. Zhao, J. Lu, R. Zhang, S.M. Yiu, K. Chow, Geminio: language-guided gradient inversion attacks in federated learning, CoRR arXiv preprint arXiv:2411.14937 (2024), https://doi.org/10.48550/ARXIV.2411.14937
[22] M.J. Mia, M.H. Amini, FedShield-LLM: a secure and scalable federated fine-tuned large language model, CoRR arXiv preprint arXiv:2506.05640 (2025), https://doi.org/10.48550/ARXIV.2506.05640